

Лекция 3. Элементы алгебры логики

Хронометраж пары (90 мин): длинное условие и таблица перевода ~7 мин → высказывание и три операции ~15 мин → таблицы истинности ~15 мин → законы ~18 мин → допуск, укорачивание и три набора ~23 мин → ДНФ, КНФ и короткая запись в Python ~12 мин.

Дополнение в конце файла (полусумматор и ключи).

Цели занятия

После лекции вы умеете:

1. Перевести правило с русского на $\neg$, $\wedge$, $\vee$ и обратно, пока перед глазами таблица обозначений.
2. Заполнить таблицу истинности для НЕ, И, ИЛИ и для выражения из двух переменных.
3. Укоротить длинное выражение: де Морган, двойное отрицание, вынесение общего члена. Сверить исходную и короткую формулу на конкретных наборах 0 и 1.
4. Узнать на готовой формуле дизъюнктивную и конъюнктивную нормальные формы.
5. Записать то же условие через `not`, `and`, `or` и не перепутать приоритет.

0. Зачем это в курсе

Условие в `if` — это высказывание. Оно либо истинно, либо ложно: один бит, 1 или 0. Пока условие короткое (`n % 2 == 0`), его видно сразу. Длинное условие из нескольких «и», «или» и «не» уже трудно прочитать, хотя каждая часть по отдельности простая.

Пример — допуск к экзамену.

- \$A\$: зачёт сдан
- \$B\$: долг по лабораторным есть
- \$C\$: преподаватель разрешил

В начале длинная запись:

```
(not (not A or B)) or (A and C)
```

В конце то же правило читается короче:

```
A and (not B or C)
```

Обозначения, которыми пишем формулы:

По-русски	Знак	В Python
истина, 1	$\$1\$$	True
ложь, 0	$\$0\$$	False
НЕ	$\$\neg\$$	not
И	$\$\wedge\$$	and
ИЛИ	$\$\vee\$$	or

Свободно вспомнить знак без таблицы пока не требуется.

1. Высказывание и три операции

Высказывание — утверждение, у которого есть ровно одно из двух значений: истина или ложь. «Зачёт сдан» — высказывание. «Сдай зачёт» и «зачёт сдан?» высказываниями не являются: у приказа и вопроса нет значения 0 или 1.

Логическая переменная — буква, вместо которой подставляют высказывание. На этой паре переменная принимает значение $\$1\$$ или $\$0\$$.

Отрицание, НЕ, $\$\neg\$$

$\$\neg X\$$ истинно ровно тогда, когда $\$X\$$ ложно.

$\$\neg B\$$ читается «долга нет», если $\$B\$$ — «долг есть». $\$\neg\neg B\$$ снова «долг есть»: два отрицания подряд снимаются. Это **двойное отрицание**: $\$\neg\neg X = X\$$.

Конъюнкция, И, $\$\wedge\$$

$\$X \wedge Y\$$ истинно только когда истинны оба. Одно ложное звено делает ложным всё «и».

$\$A \wedge \neg B\$$ — «зачёт сдан и долга нет». Если зачёт сдан, а долг есть, значение $\$1 \wedge \neg 1 = 1 \wedge 0 = 0\$$.

Дизъюнкция, ИЛИ, $\$\vee\$$

$\$X \vee Y\$$ истинно, когда истинно хотя бы одно. **Включительное «или»**: если истинны оба, результат всё равно истина.

$\$1 \vee 1 = 1\$$

«Зачёт сдан или преподаватель разрешил» остаётся истиной и когда выполнены оба условия сразу. Бытовое «или» иногда понимают как «ровно одно из двух».

Приоритет

Как в Python: сначала $\neg$, затем $\wedge$, затем $\vee$. Это те же ступени, что у `not`, `and`, `or`.

$\neg A \vee B$ — это $(\neg A) \vee B$, не $\neg(A \vee B)$.

Скобки ставим, когда нужный порядок слабее приоритета. Их же ставим, когда без них формулу легко прочесть не туда, даже если приоритет и так верный.

Частая ошибка. Отрицание вешают на всю формулу, хотя по записи оно относится только к ближайшему члену. Вторая ошибка: бытовое «или» в смысле «либо одно, либо другое, но не оба». Для $1 \vee 1$ это уже другая функция.

*В Python `True or что-то ещё` иногда не вычисляет второй операнд.

2. Таблицы истинности

Таблица истинности перечисляет все наборы нулей и единиц и значение выражения на каждом наборе. Число строк для n переменных — 2^n . Одна переменная даёт 2 строки, две — 4, три — 8.

Три операции

X	$\neg X$
0	1
1	0

X	Y	$X \wedge Y$
0	0	0
0	1	0
1	0	0
1	1	1

X	Y	$X \vee Y$
0	0	0
0	1	1
1	0	1
1	1	1

Выражение из двух переменных

$A \wedge \neg B$: «зачёт сдан и долга нет». Сначала столбец $\neg B$, потом «и».

A	B	$\neg B$	$A \wedge \neg B$
0	0	1	0
0	1	0	0
1	0	1	1
1	1	0	0

Единица в одной строке: зачёт есть, долга нет. В остальных трёх допуск по этому короткому правилу не проходит.

Частая ошибка. Пропуск строки. Для двух переменных обязательны все четыре пары: 00, 01, 10, 11. Вторая ошибка: отрицание применяют к уже посчитанному «и», хотя в формуле $\neg$ стоит только над B .

3. Законы, которыми укорачивают

Две формулы **равны**, если на каждом наборе нулей и единиц у них одно и то же значение. Укорачивание не меняет функцию: меняется запись. Проверка — таблица или подстановка конкретных наборов.

Де Морган

Отрицание «и» — это «или» отрицаний. Отрицание «или» — это «и» отрицаний.

$$\neg(X \wedge Y) = \neg X \vee \neg Y$$

$$\neg(X \vee Y) = \neg X \wedge \neg Y$$

Проверка для, $X = 1$, $Y = 0$:

- слева $\neg(1 \wedge 0) = \neg 0 = 1$
- справа $\neg 1 \vee \neg 0 = 0 \vee 1 = 1$

«Неверно, что зачёт сдан и долг есть» значит «зачёта нет или долга нет». Это не то же самое, что «зачёта нет и долга нет».

Проверка остальных вариантов, самостоятельно.

Двойное отрицание

$$\neg\neg X = X$$

Коммутативность

$$X \wedge Y = Y \wedge X, \quad X \vee Y = Y \vee X$$

Порядок условий в «и» и в «или» не меняет значение. «Долга нет или преподаватель разрешил» и «преподаватель разрешил или долга нет» — одна функция. Ассоциативность такая же, как у сложения и умножения чисел: $(X \wedge Y) \wedge Z = X \wedge (Y \wedge Z)$, и так же для $\vee$.

Ноль, единица и повтор

$$X \wedge 1 = X, \quad X \vee 0 = X, \quad X \wedge X = X, \quad X \vee X = X$$

«Зачёт сдан и истина» ничего не добавляет к «зачёт сдан». «Долг есть или долг есть» — по-прежнему один долг. Эти четыре равенства как раз вычёркивают лишнее из длинной записи.

Вынесение общего члена

$$(X \wedge Y) \vee (X \wedge Z) = X \wedge (Y \vee Z)$$

Общее условие выносят за скобку, запись короче. Раскрытие скобок идёт в другую сторону и запись обычно длиннее:

$$X \wedge (Y \vee Z) = (X \wedge Y) \vee (X \wedge Z)$$

Проверка вынесения: $X = 1$, $Y = 0$, $Z = 1$. Слева $(1 \wedge 0) \vee (1 \wedge 1) = 0 \vee 1 = 1$. Справа $1 \wedge (0 \vee 1) = 1$.

Частая ошибка. $\neg(X \wedge Y)$ превращают в $\neg X \wedge \neg Y$. Знак между членами тоже меняется. Вторая ошибка: ради «упрощения» скобки раскрывают и получают более длинную запись. Укорачивание это вынесение общего члена, снятие двойного отрицания, вычёркивание повтора и нуля с единицей.

В справочнике в конце есть ещё поглощение и исключённое третье.

4. Допуск: длинное выражение

Фраза:

Допуск есть, если неверно, что зачёт не сдан или долг есть, или если зачёт сдан и преподаватель разрешил.

«Неверно, что» относится только к куску «зачёт не сдан или долг есть». Дальше отдельное «или если».

$$\neg(\neg A \vee B) \vee (A \wedge C)$$

Укорачивание

Де Морган ко внутренней скобке, затем двойное отрицание:

$$\neg(\neg A \vee B) = \neg\neg A \wedge \neg B = A \wedge \neg B$$

Формула становится дизъюнкцией двух конъюнкций:

$$(A \wedge \neg B) \vee (A \wedge C)$$

Общий член A выносим:

$$A \wedge (\neg B \vee C)$$

По-русски: «зачёт сдан и (долга нет или преподаватель разрешил)».

Три набора

Подставляем и в длинную формулу, и в короткую.

Долг есть, разрешения нет: $A = 1, B = 1, C = 0$.

- длинная: $\neg(\neg 1 \vee 1) \vee (1 \wedge 0) = \neg(0 \vee 1) \vee 0 = 0$
- короткая: $1 \wedge (\neg 1 \vee 0) = 1 \wedge 0 = 0$

Допуска нет.

Долг есть, преподаватель разрешил: $A = 1, B = 1, C = 1$.

- длинная: $\neg(0 \vee 1) \vee (1 \wedge 1) = 0 \vee 1 = 1$
- короткая: $1 \wedge (0 \vee 1) = 1$

Допуск есть: срабатывает вторая часть «или».

Зачёта нет, долга нет, преподаватель разрешил: $A = 0, B = 0, C = 1$.

- длинная: $\neg(\neg 0 \vee 0) \vee (0 \wedge 1) = \neg(1 \vee 0) \vee 0 = 0$
- короткая: $0 \wedge (1 \vee 1) = 0$

Допуска нет. Разрешение преподавателя само по себе зачёт не заменяет.

Частая ошибка. «Неверно, что» распространяют на всё предложение и получают одно большое отрицание. Вторая ошибка: после де Моргана забывают снять $\neg\neg A$.

5. ДНФ, КНФ и строка Python

Короткая формула допуска уже собрана как «и» с «или» внутри:

$$A \wedge (\neg B \vee C)$$

Раскроем скобку:

$$(A \wedge \neg B) \vee (A \wedge C)$$

Дизъюнктивная нормальная форма (ДНФ) — дизъюнкция конъюнкций. Правая запись выше как раз такая: два «и», соединённые «или».

Конъюнктивная нормальная форма (КНФ) — конъюнкция дизъюнкций. Запись $A \wedge (\neg B \vee C)$ такая: снаружи «и», внутри «или». Одиночное A считают дизъюнкцией из одного члена.

Обе записи задают одну и ту же функцию. ДНФ здесь длиннее. Укорачивание, которое мы делали, привело к КНФ. Имя форме даём после того, как формула уже укорочена. Отдельный вывод «любой формулы к ДНФ» на этой паре не учим.

Теорема Любая булева функция, отличная от константы, представима в виде ДНФ и КНФ.

Таблица на восемь строк

A	B	C	$\neg(\neg A \vee B) \vee (A \wedge C)$	$A \wedge (\neg B \vee C)$
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	0	0
1	0	0	1	1
1	0	1	1	1
1	1	0	0	0
1	1	1	1	1

Строка $A = 1, B = 0, C = 1$: истинны обе части исходного «или» сразу. Включительное «или» даёт 1, и короткая формула тоже даёт 1.

Та же функция в Python

```
A and (not B or C)
```

`not` сильнее `or`, поэтому `not B or C` — это `(not B) or C`. Скобки вокруг этого куска нужны из-за `and`: без них запись читается иначе.

Частая ошибка.

```
A and not B or C
```

Приоритет `and` выше `or`, поэтому это `(A and not B) or C`. При $A = 0, B = 0, C = 1$ такая запись даёт 1: человека пускают без зачёта. Верная формула $A \wedge (\neg B \vee C)$ на этом наборе даёт 0.

Как из тех же операций собирают сложение одного бита, написано в дополнении в конце файла.

Справочник

Обозначения

По-русски	Знак	В Python
истина	$\$1\$$	<code>True</code>
ложь	$\$0\$$	<code>False</code>
НЕ	$\$\neg\$$	<code>not</code>
И	$\$\wedge\$$	<code>and</code>
ИЛИ	$\$\vee\$$	<code>or</code>

Приоритет: $\$\neg\$$, затем $\$\wedge\$$, затем $\$\vee\$$. В Python так же: `not`, затем `and`, затем `or`.

Законы основные

- Двойное отрицание: $\$\neg\neg X = X\$$.
- Де Морган: $\$\neg(X \wedge Y) = \neg X \vee \neg Y\$$ и $\$\neg(X \vee Y) = \neg X \wedge \neg Y\$$.
- Коммутативность: $\$X \wedge Y = Y \wedge X\$$, $\$X \vee Y = Y \vee X\$$.
- Ноль, единица, повтор: $\$X \wedge 1 = X\$$, $\$X \vee 0 = X\$$, $\$X \wedge X = X\$$, $\$X \vee X = X\$$.
- Вынесение: $\$(X \wedge Y) \vee (X \wedge Z) = X \wedge (Y \vee Z)\$$.

Законы только для чтения

Ассоциативность: $\$(X \wedge Y) \wedge Z = X \wedge (Y \wedge Z)\$$, так же для $\$\vee\$$.

Исключённое третье и противоречие: $\$X \vee \neg X = 1\$$, $\$X \wedge \neg X = 0\$$.

Поглощение: $\$X \vee (X \wedge Y) = X\$$, $\$X \wedge (X \vee Y) = X\$$.

Нормальные формы

ДНФ — дизъюнкция конъюнкций. КНФ — конъюнкция дизъюнкций.

На примере допуска КНФ: $\$A \wedge (\neg B \vee C)\$$. ДНФ той же функции: $\$(A \wedge \neg B) \vee (A \wedge C)\$$.

Упражнения

Сначала попробуйте сами, эталоны в следующем разделе. Таблица обозначений из справочника пусть лежит рядом: так и задумано.

1. Перевод. «Зачёт сдан и преподаватель разрешил.» Запишите знаками.

2. Перевод. $\neg B \vee C$. Запишите по-русски, по легенде допуска.

3. Перевод. «Неверно, что долг есть или зачёта нет.» Запишите знаками, скобки не теряйте.

4. Перевод. $A \wedge \neg B$. Запишите через `and`, `or`, `not`.

5. Перевод. `not A or B`. Запишите знаками и по-русски.

6. Найди ошибку. Утверждают, что $\neg(A \vee B) = \neg A \vee \neg B$. Найдите строку таблицы, где значения расходятся. Напишите верный закон де Моргана для этого случая и проверьте им ту же строку.

7. Второе правило. Напоминание отправляют, если неверно, что работа сдана или дедлайн нет, или если дедлайн завтра и работа не сдана.

- D : дедлайн завтра
- W : работа сдана

Переведите, укоротите и проверьте два набора: дедлайн завтра и работа уже сдана; дедлайн завтра и работа не сдана.

Ответы к упражнениям

1. $A \wedge C$.

2. «Долга нет или преподаватель разрешил.»

3. $\neg(B \vee \neg A)$. После де Моргана и двойного отрицания это же $A \wedge \neg B$, но в ответе на перевод достаточно формулы со скобкой.

4. `A and not B`. Скобки не обязательны: `not` сильнее `and`.

5. $\neg A \vee B$. «Зачёта нет или долг есть.» Это не $\neg(A \vee B)$.

6. Строка $A = 1$, $B = 0$: слева $\neg(1 \vee 0) = 0$, справа $\neg 1 \vee \neg 0 = 1$. Верно $\neg(A \vee B) = \neg A \wedge \neg B$. На той же строке: $\neg 1 \wedge \neg 0 = 0$.

7. $\neg(W \vee \neg D) \vee (D \wedge \neg W)$. Де Морган и двойное отрицание дают $(\neg W \wedge D) \vee (D \wedge \neg W)$. Коммутативность выравнивает члены: $(D \wedge \neg W) \vee (D \wedge \neg W)$. Повтор вычёркивается: $D \wedge \neg W$. «Дедлайн завтра и работа не сдана.»

Набор $D = 1$, $W = 1$: и длинная, и короткая формулы дают 0. Напоминание не шлём. Набор $D = 1$, $W = 0$: обе дают 1. Напоминание шлём.

Контрольные вопросы

Короткие вопросы на понимание. Эталоны в следующем разделе; сначала ответьте сами.

- 1. Чему равно $1 \vee 1$ и почему для условия в `if` это важно?
- 2. Как прочитать $\neg(A \vee B)$ и чем это отличается от $\neg(A \vee \neg B)$?
- 3. Сформулируйте оба закона де Моргана словами «и», «или», «не».
- 4. Зачем после укорачивания подставлять конкретные 0 и 1 в обе формулы?
- 5. Какая из записей допуска является ДНФ, а какая КНФ: $A \wedge (\neg B \vee C)$ и $(A \wedge \neg B) \vee (A \wedge C)$?
- 6. Чем отличаются `A and (not B or C)` и `A and not B or C`? Приведите набор, на котором они расходятся.

Ответы для самопроверки

- 1. $1 \vee 1 = 1$. Если истинны оба условия, ветка `if` выполняется. Это не «ровно одно из двух».
- 2. $\neg(A \vee B)$ — «зачёта нет или долг есть»: отрицание только над A . $\neg(A \vee \neg B)$ — «неверно, что зачёт сдан или долг есть», по де Моргану $\neg A \wedge \neg \neg B$.
- 3. Отрицание «и» равно «или» отрицаний. Отрицание «или» равно «и» отрицаний.
- 4. Равные формулы дают одно значение на каждом наборе. Подстановка ловит ошибку в законе: скобку раскрыли не туда или не сменили «и» на «или».
- 5. $(A \wedge \neg B) \vee (A \wedge C)$ — ДНФ. $A \wedge (\neg B \vee C)$ — КНФ.
- 6. Первая формула — $A \wedge (\neg B \vee C)$. Вторая из-за приоритета — $(A \wedge \neg B) \vee C$. Набор $A = 0, B = 0, C = 1$: верная даёт 0, ошибочная даёт 1.

Дополнение. Полусумматор и ключи

Компьютер складывает числа по битам. Возьмём два однобитных числа x и y . Это не буквы допуска: здесь x и y — разряды.

Сумма $1 + 1$ в один бит не помещается. Схема выдаёт два бита: **сумму** в текущем разряде и **перенос** в следующий. Такая схема для двух битов без входящего переноса называется **полусумматором**.

x	y	перенос	сумма
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Перенос — конъюнкция:

$$s = x \wedge y$$

Сумма — дизъюнкция двух конъюнкций, то есть ДНФ:

$$s = (x \wedge \neg y) \vee (\neg x \wedge y)$$

Единица на выходе суммы стоит, когда единицы различны. Когда единицы совпали, сумма в этом разряде равна 0, а единица уходит в перенос. Эту функцию называют **исключающим или**. Её не было среди трёх операций пары: у обычного $\vee$ клетка $1 \vee 1$ равна 1, а у суммы двух битов в этой клетке 0.

Транзистор здесь — ключ

В этой схеме транзистор — управляемый ключ. Входной бит решает, замкнут ключ или разомкнут. Сам ключ ещё не «и», не «или» и не «не». Булеву операцию считает **вентиль**: несколько ключей, собранных в одну схему.

Три сборки, которых хватает для полусумматора.

И. Два ключа один за другим. Лампа горит, только если замкнуты оба. Это $x \wedge y$. Так устроен перенос.

питание — [ключ x] — [ключ y] — лампа

ИЛИ. Два ключа рядом, параллельно. Лампа горит, если замкнут хотя бы один. Это $x \vee y$.

```

питание —+— [ключ x] —+— лампа
           |               |
           +— [ключ y] —+
  
```

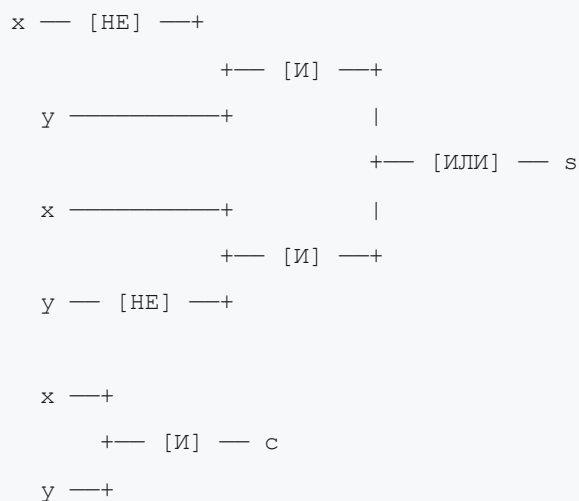
НЕ. Ключ размыкающий: при 1 на входе цепь рвётся, при 0 она замкнута. Лампа горит, когда на входе 0. Это $\neg x$.

питание — [ключ x размыкается при 1] — лампа

Схема полусумматора из вентилях

Перенос — один вентиль И.

Сумма собирается из тех же вентилях. Сначала два НЕ, затем два И, затем одно ИЛИ:



Каждый прямоугольник — вентиль, то есть уже собранные ключи, а не отдельный транзистор. В реальности всё чуть сложнее, но принцип тот же.

Полупроводники, КМОП, исток и сток, полный сумматор с входящим переносом и сложение многобитного числа в эту страницу не входят.